마이크로프로세서

Chapter 4. GPIO & EXTI

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY

GPIO

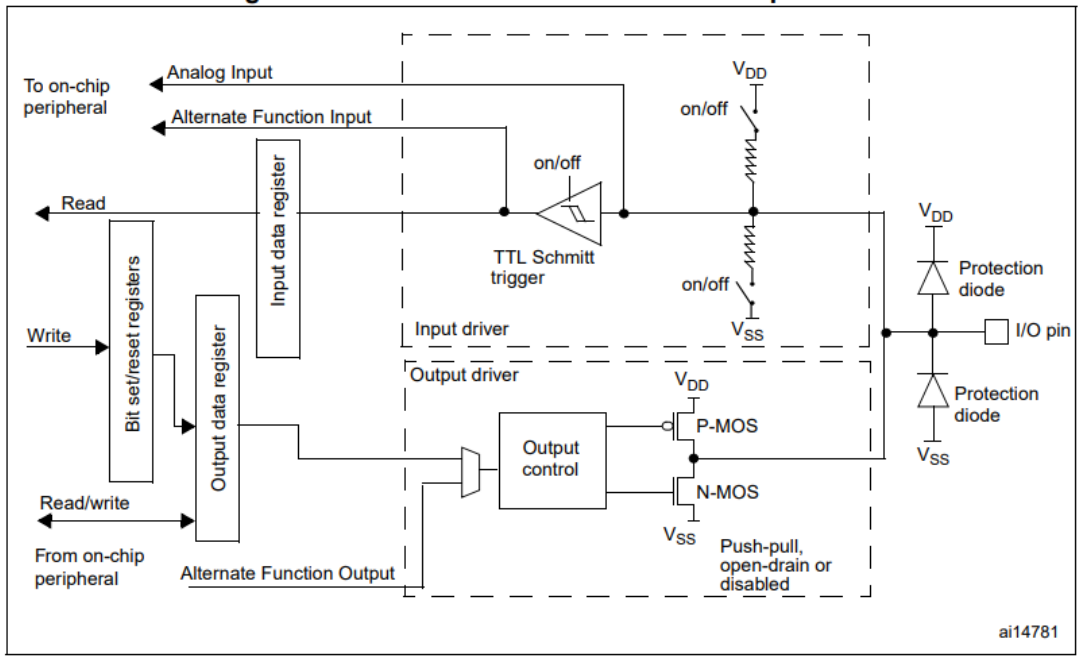
>> General-Purpose Input/Output

- 범용 입출력(General-Purpose Input/Output)은 프로세서의 설정을 통해 핀의 입력/출력 기능을 제공
  - 입력/출력모드: 디지털 전압 출력 / 디지털 전압 측정(입력)
  - Alternate Function: 통신(UART, I2C, SPI …), PWM, MCO 등 특수 입출력 모드 설정
  - 아날로그 모드: ADC 입력, DAC 출력 모드
  - EXIT: External Interrupt 외부 인터럽트

- GPIO 출력 모드
  - Push-Pull: HIGH/LOW 출력
  - Open-Drain: LOW만 출력, HIGH출력시 풀업 저항 필

- GPIO 입력 모드
  - Pull-up: 기본 HIGH 유지
  - Pull-down: 기본 LOW 유지

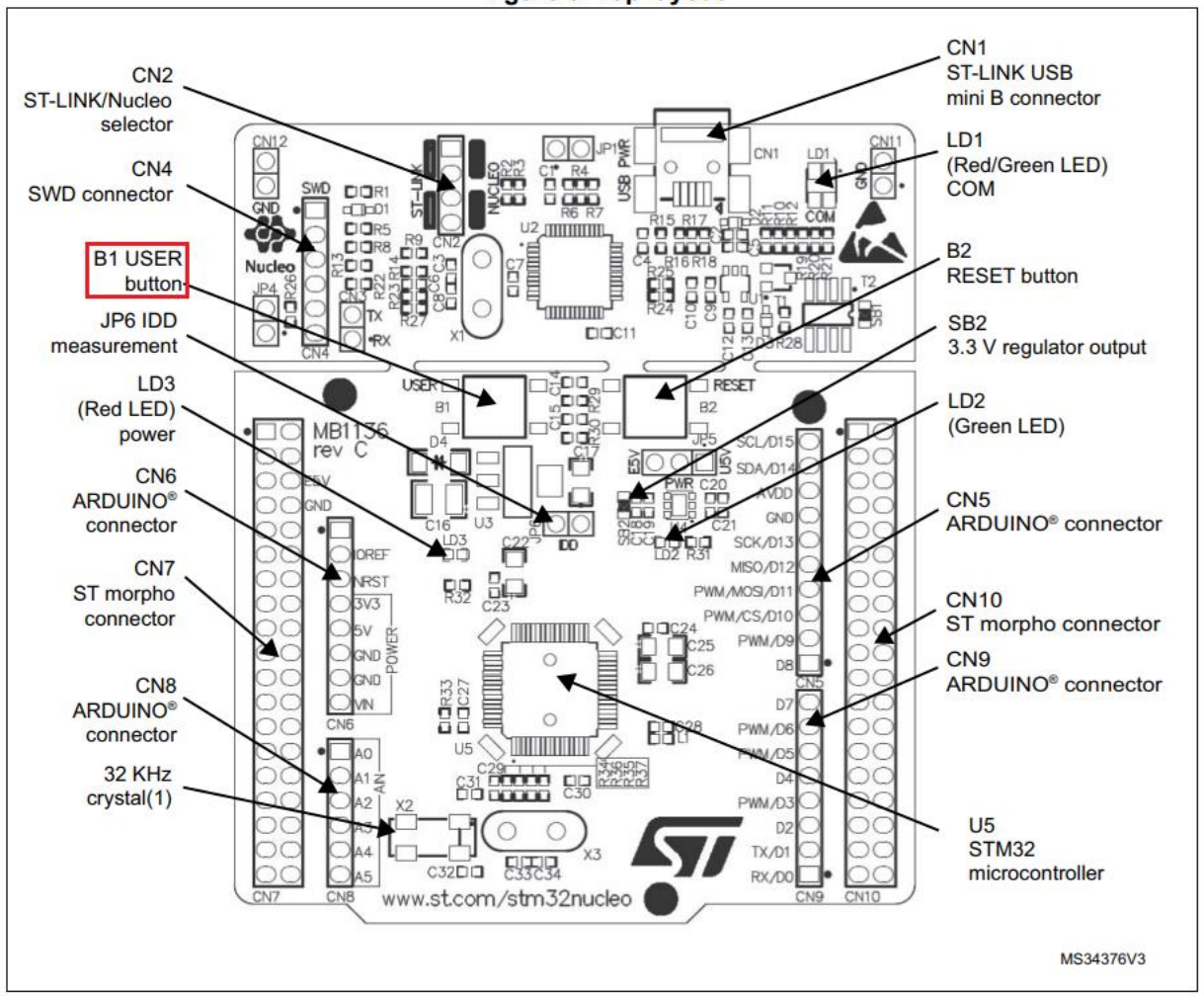
Figure 13. Basic structure of a standard I/O port bit

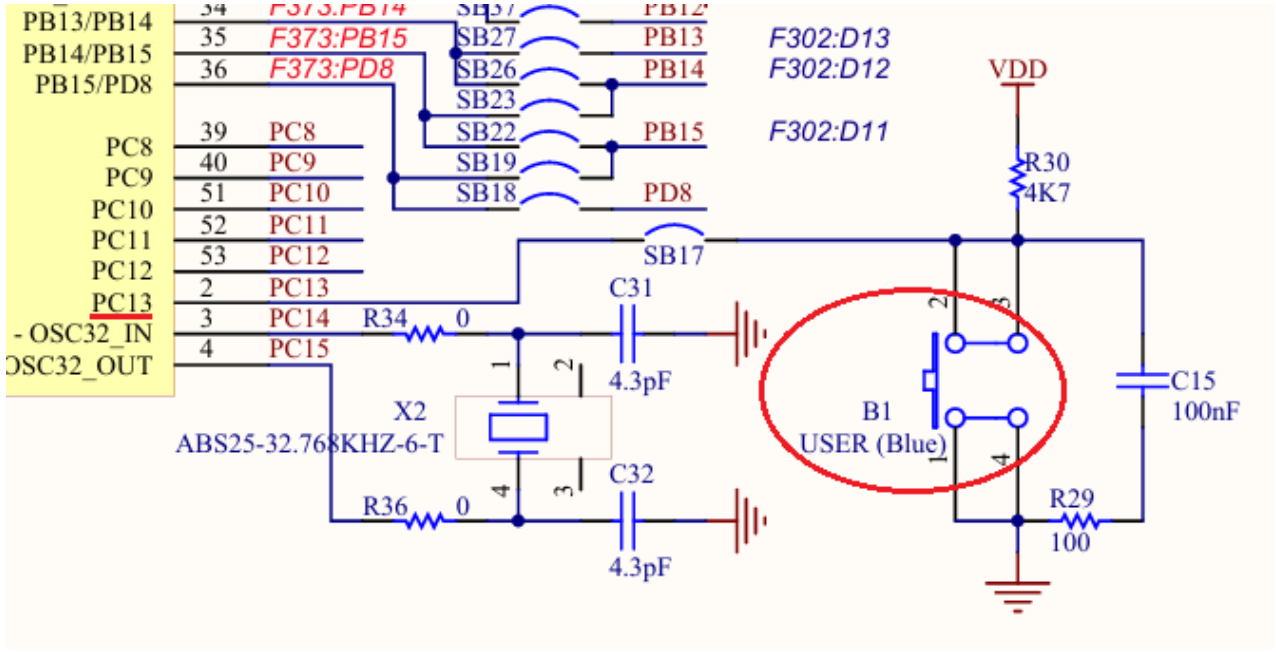


GPIO

>> General-Purpose Input/Output

Figure 3. Top layout

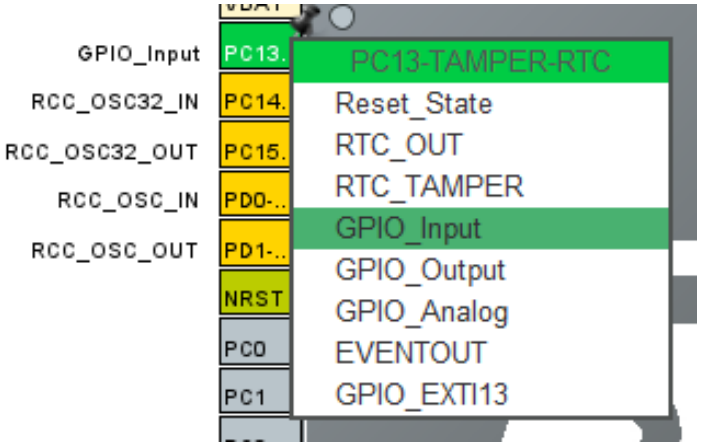


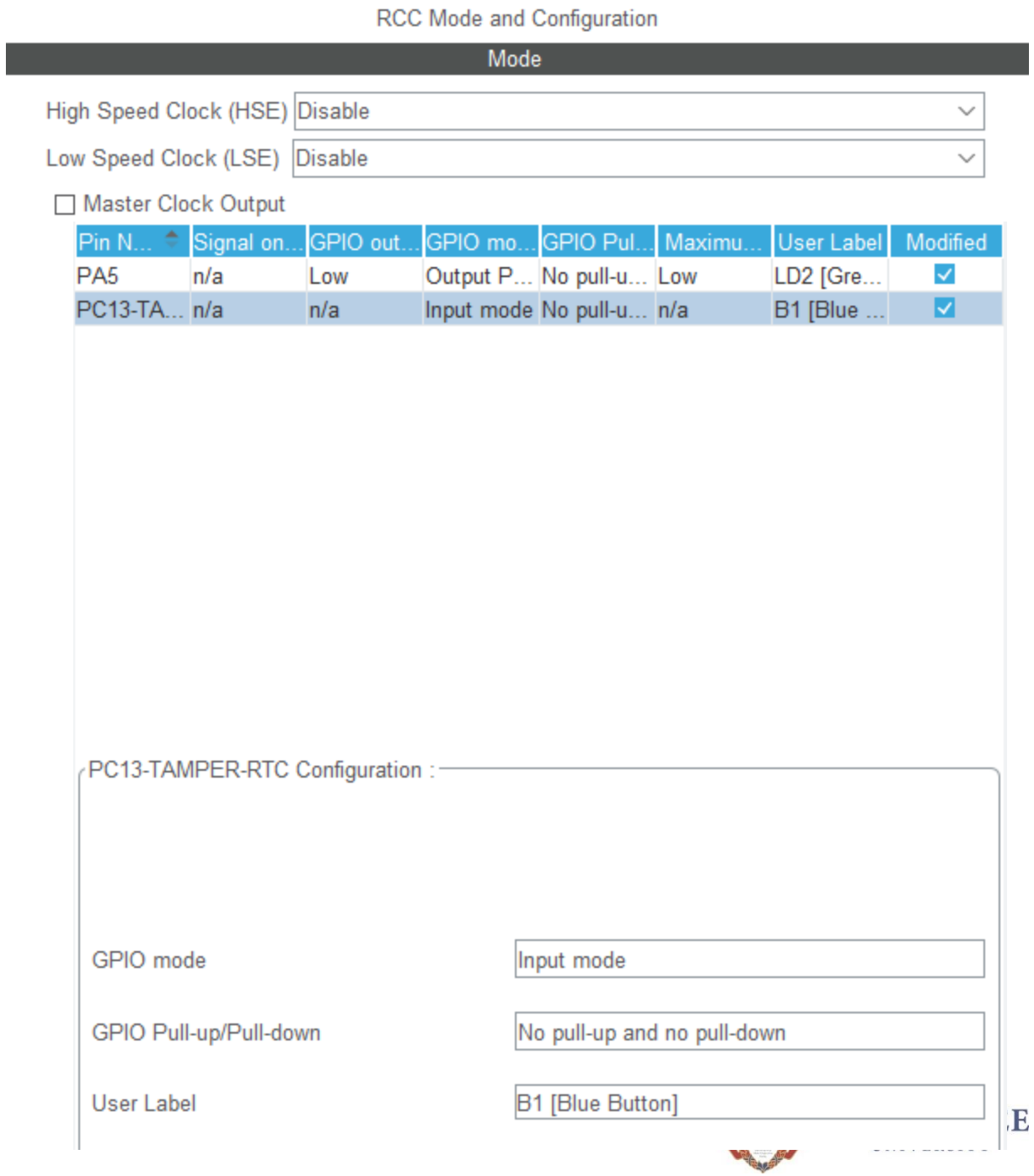


GPIO

>> General-Purpose Input/Output

- **PC13을 GPIO_Input으로 설정**





GPIO

>> General-Purpose Input/Output

- main.h에서 Label 적용 확인
- main.c while 부분 USER CODE에 작성

```
#define B1_Pin GPIO_PIN_13
#define B1_GPIO_Port GPIOC
#define USART_TX_Pin GPIO_PIN_2
#define USART_TX_GPIO_Port GPIOA
#define USART_RX_Pin GPIO_PIN_3
#define USART_RX_GPIO_Port GPIOA
#define LD2_Pin GPIO_PIN_5
#define LD2_GPIO_Port GPIOA
#define TMS_Pin GPIO_PIN_13
#define TMS_GPIO_Port GPIOA
#define TCK_Pin GPIO_PIN_14
#define TCK_GPIO_Port GPIOA
#define SWO_Pin GPIO_PIN_3
#define SWO_GPIO_Port GPIOB
```

main.h

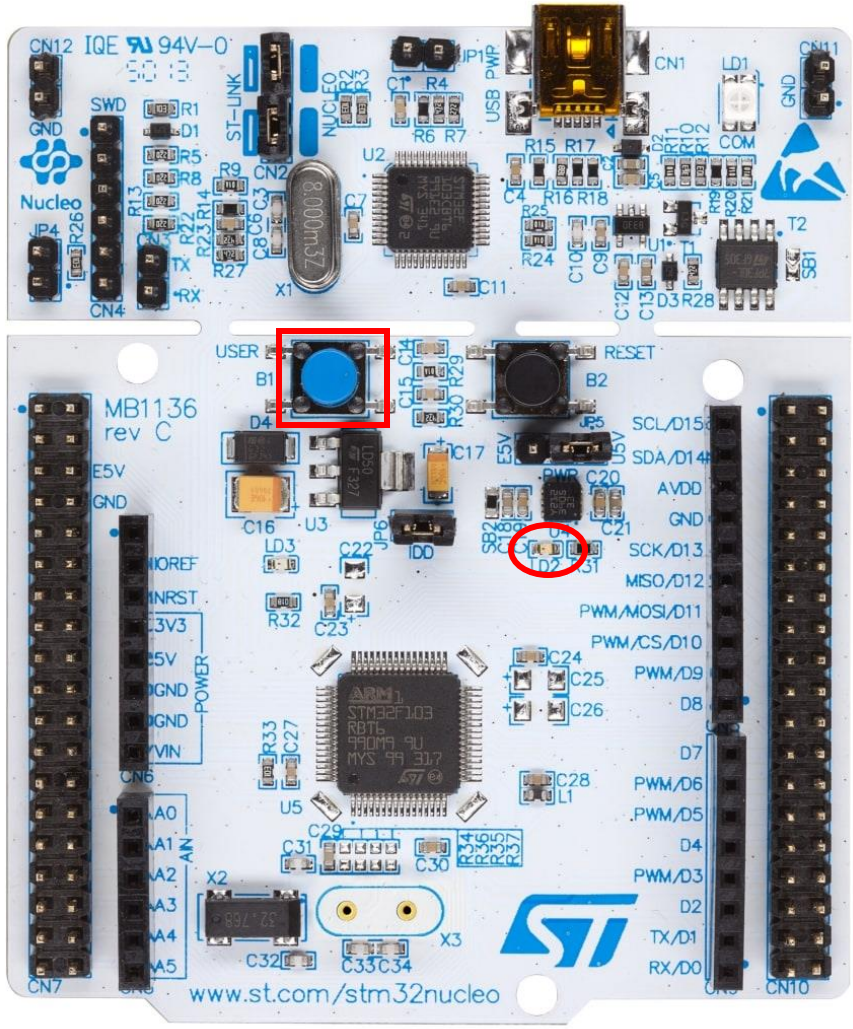
```
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
      if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin))
          HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
      else
          HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
  }
  /* USER CODE END 3 */
}
```

main.c

GPIO

>> General-Purpose Input/Output

- 버튼에 따른 LED 동작 확인
- 버튼을 누르는 경우?
- 버튼을 누르지 않은 경우?

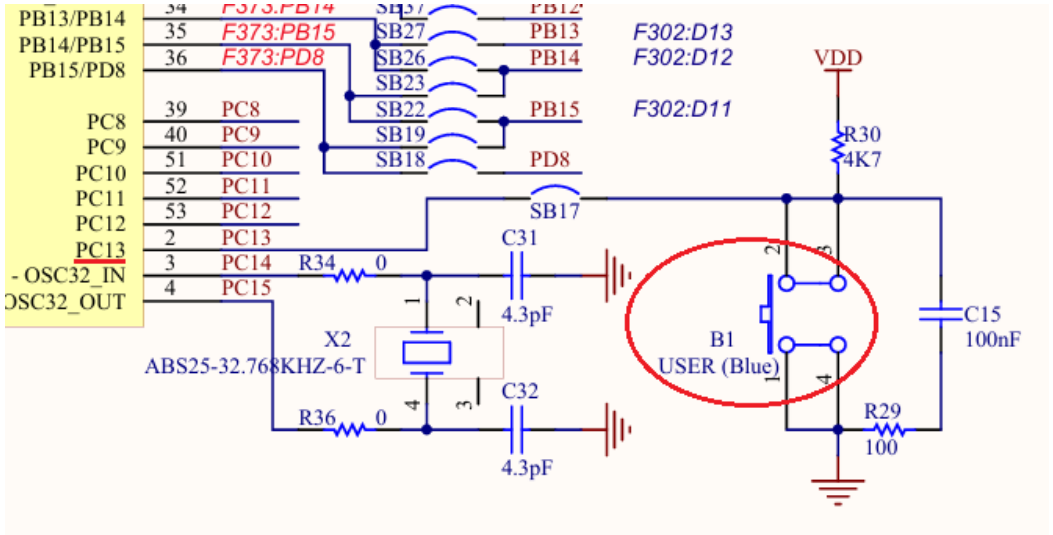


GPIO

>> General-Purpose Input/Output

- CubeMX에서 No Pull-up, No Pull-down 모드로 설정
- 해당 모드는 입력 핀이 Floating 상태로 HIGH/LOW 상태 결정되어 있지 않음
- B1 버튼은 보드에서 기본적으로 Pull-Up 회로가 구성되어 있음
- 따라서 버튼을 누르면 LOW/누르지 않으면 HIGH

GPIO mode	Input mode
GPIO Pull-up/Pull-down	No pull-up and no pull-down
User Label	B1 [Blue Botton]



GPIO

>> General-Purpose Input/Output

- GPIO 입력은 코드 중간에 명령어를 통해 입력 핀의 값을 수동적으로 읽어와야 함
- `HAL_GPIO_ReadPin`은 input data register(IDR)을 읽음으로써 입력 핀의 상태 비트를 활용함
- 이는 프로세서가 능동적으로 외부의 변화에 대응하기에 부적합
- 인터럽트를 활용하면 프로세서가 현재 실행중인 코드의 순서를 가로챌 수 있음 (Context Switching)

```c
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
{
  GPIO_PinState bitstatus;

  /* Check the parameters */
  assert_param(IS_GPIO_PIN(GPIO_Pin));

  if ((GPIOx->IDR & GPIO_Pin) != (uint32_t)GPIO_PIN_RESET)
  {
    bitstatus = GPIO_PIN_SET;
  }
  else
  {
    bitstatus = GPIO_PIN_RESET;
  }
  return bitstatus;
}
```

Stm32f1xx_hal_gpio.c

9.2.3 Port input data register (GPIOx_IDR) (x=A..G)

Address offset: 0x08h

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

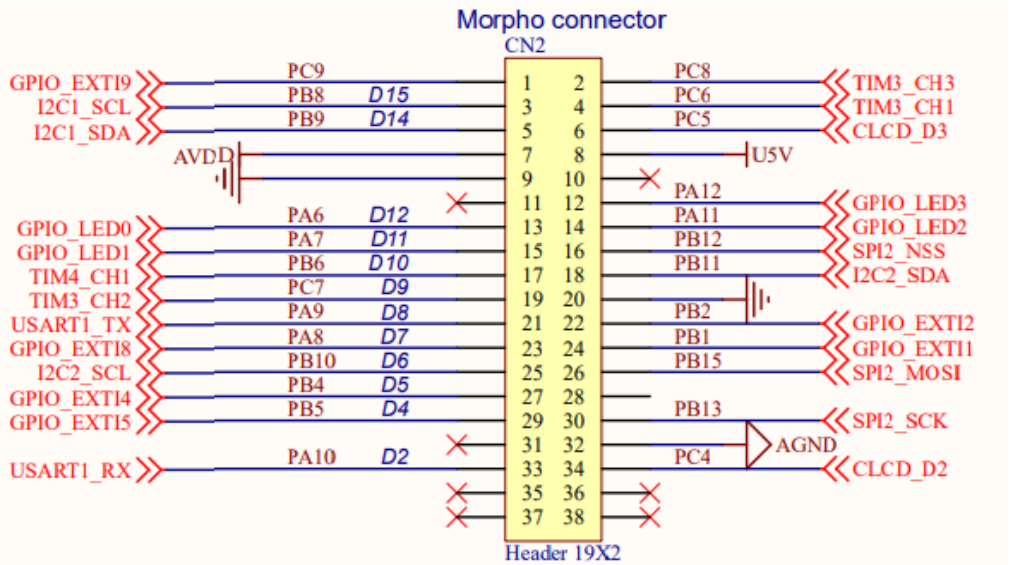
Bits 15:0 **IDRy:** Port input data (y= 0 .. 15)

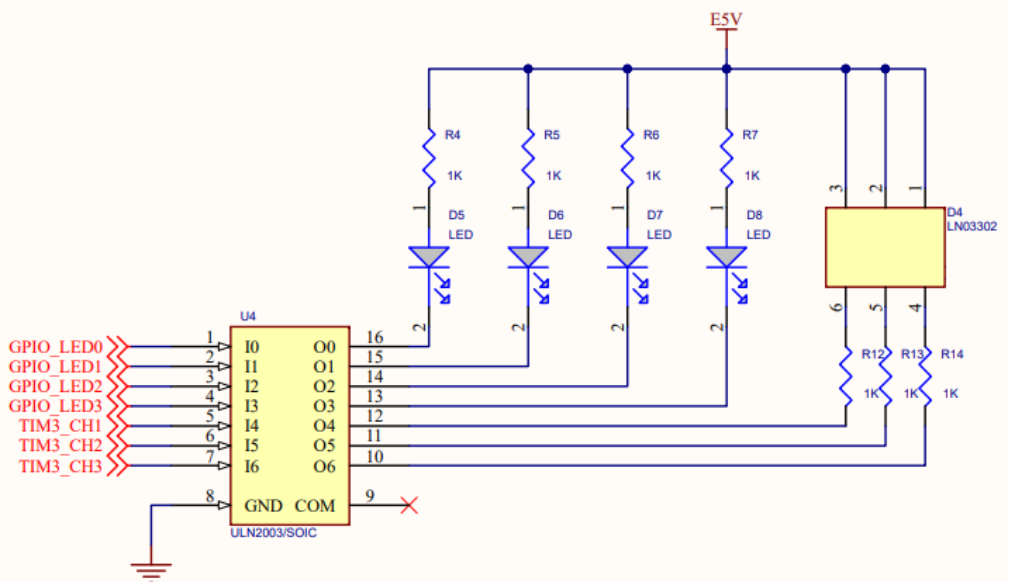
These bits are read only and can be accessed in Word mode only. They contain the input value of the corresponding I/O port.

GPIO

>> GPIO 실습

- **확장 보드 실습 – LED & Button**

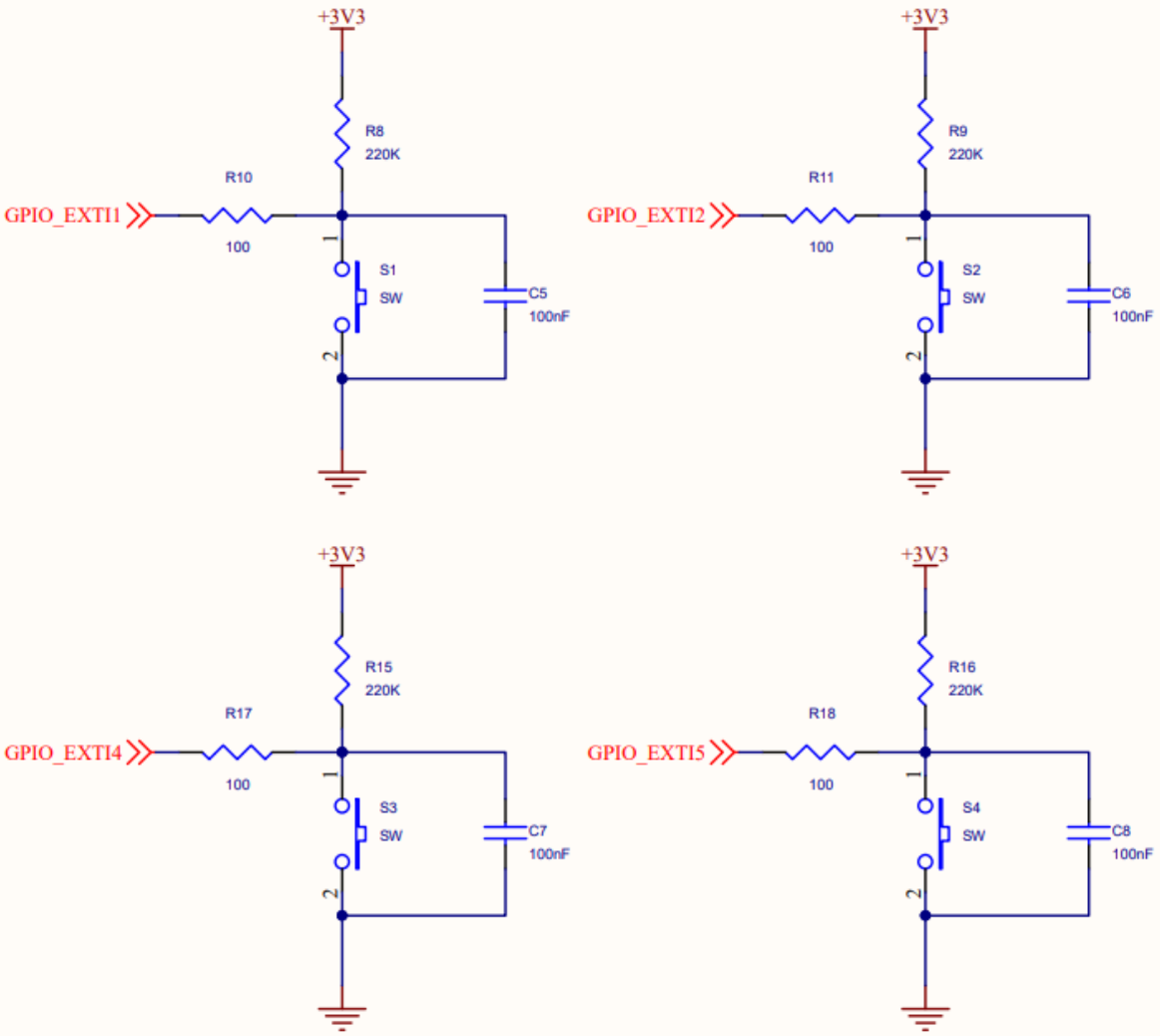




KYUNG HEE UNIVERSITY

GPIO

>> GPIO 실습

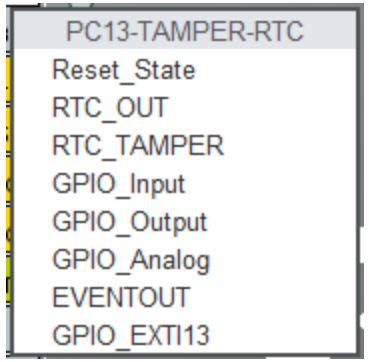


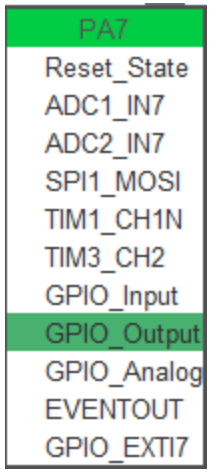
User Button

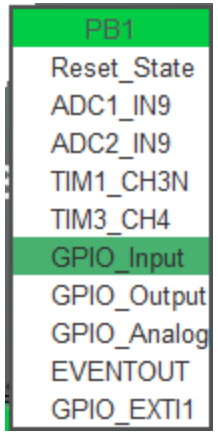
GPIO

>> GPIO 실습

- Reset_State로 핀에 할당된 설정 제거
- PA6,7,11,12 -> GPIO_Output
- PB1,2,3,4 -> GPIO_Input







Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/P...	Maximum outpu...	User Label	Modified
PA6	n/a	Low	Output Push Pull	No pull-up and ...	Low	LED1	☑
PA7	n/a	Low	Output Push Pull	No pull-up and ...	Low	LED2	☑
PA11	n/a	Low	Output Push Pull	No pull-up and ...	Low	LED3	☑
PA12	n/a	Low	Output Push Pull	No pull-up and ...	Low	LED4	☑
PB1	n/a	n/a	Input mode	No pull-up and ...	n/a	B1	☑
PB2	n/a	n/a	Input mode	No pull-up and ...	n/a	B2	☑
PB4	n/a	n/a	Input mode	No pull-up and ...	n/a	B3	☑
PB5	n/a	n/a	Input mode	No pull-up and ...	n/a	B4	☑

GPIO

>> GPIO 실습

- main.c에 다음 코드 작성 후 업로드
- 보드 좌상단 DC잭에 전원을 넣어 확장 보드에 전원 공급
- 각 버튼을 하나씩 눌러보면서 동작 확인
- 여러 버튼을 눌러보면서 동작 확인

```c
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
    if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin))
            HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);

    if (HAL_GPIO_ReadPin(B1_GPIO_Port, B2_Pin))
            HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_SET);

    if (HAL_GPIO_ReadPin(B1_GPIO_Port, B3_Pin))
            HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_SET);

    if (HAL_GPIO_ReadPin(B1_GPIO_Port, B4_Pin))
            HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_SET);
}
```

>> GPIO 실습

- 단일 토글 모드 터치 센서, 터치로 인한 정전 용량 변화시 출력 핀 토글

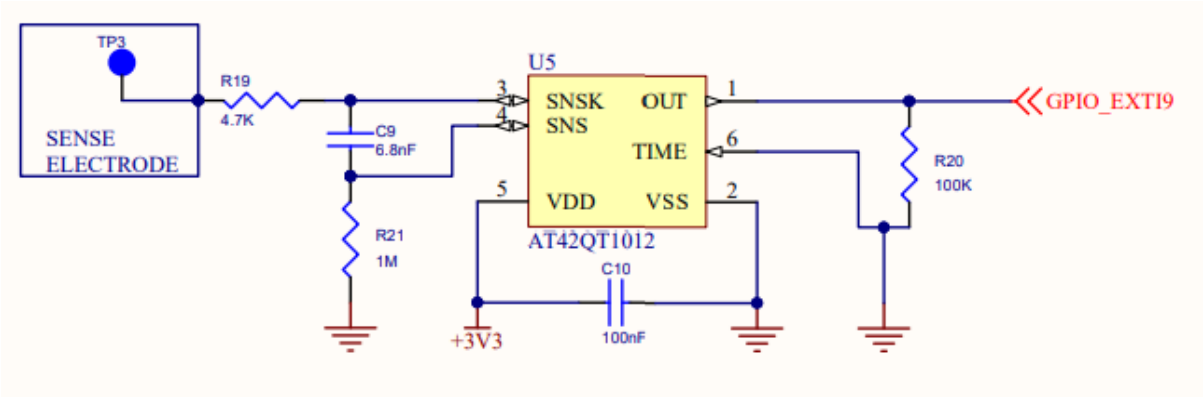
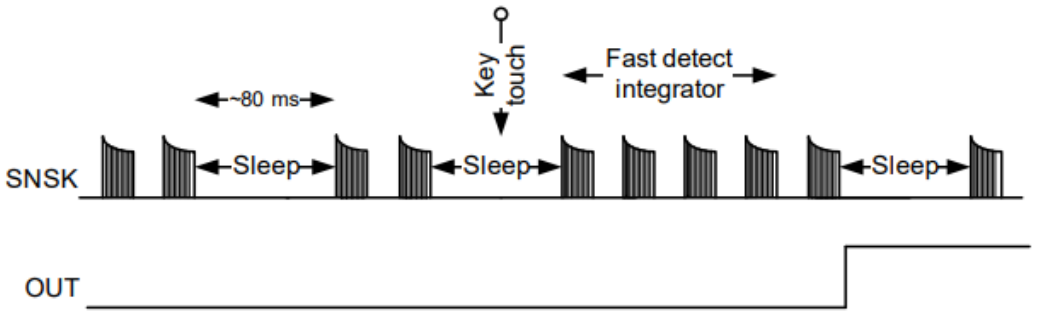
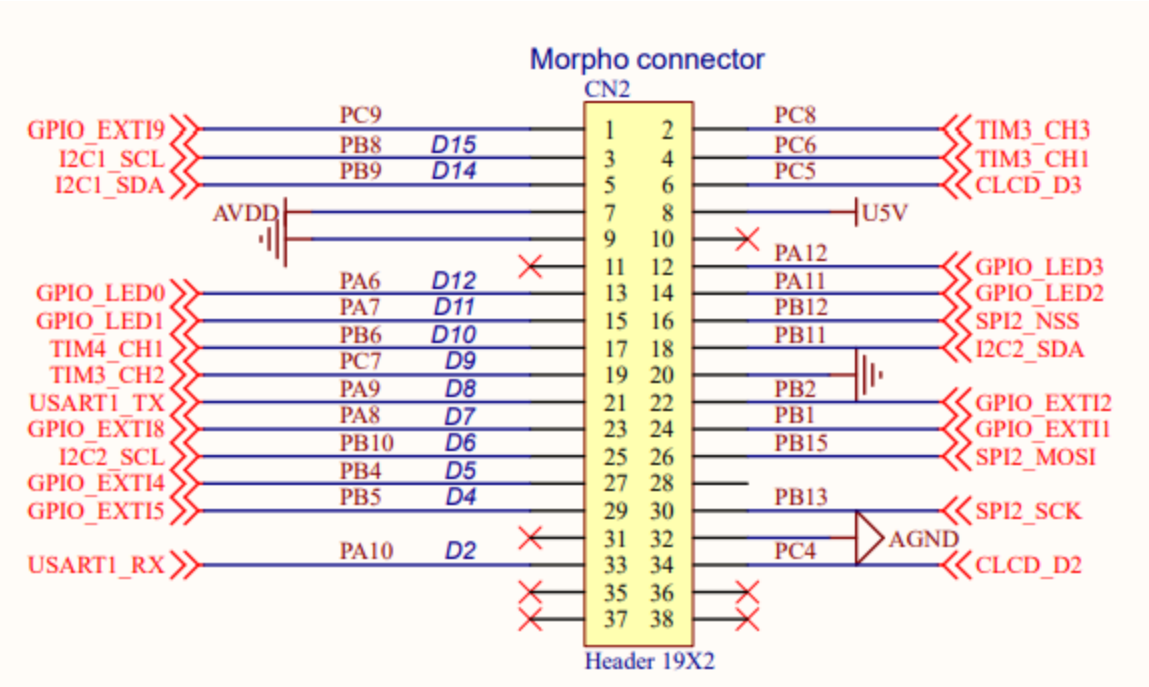


Figure 3-1. Low Power Mode: Touch Confirmed (Output in Off Condition)

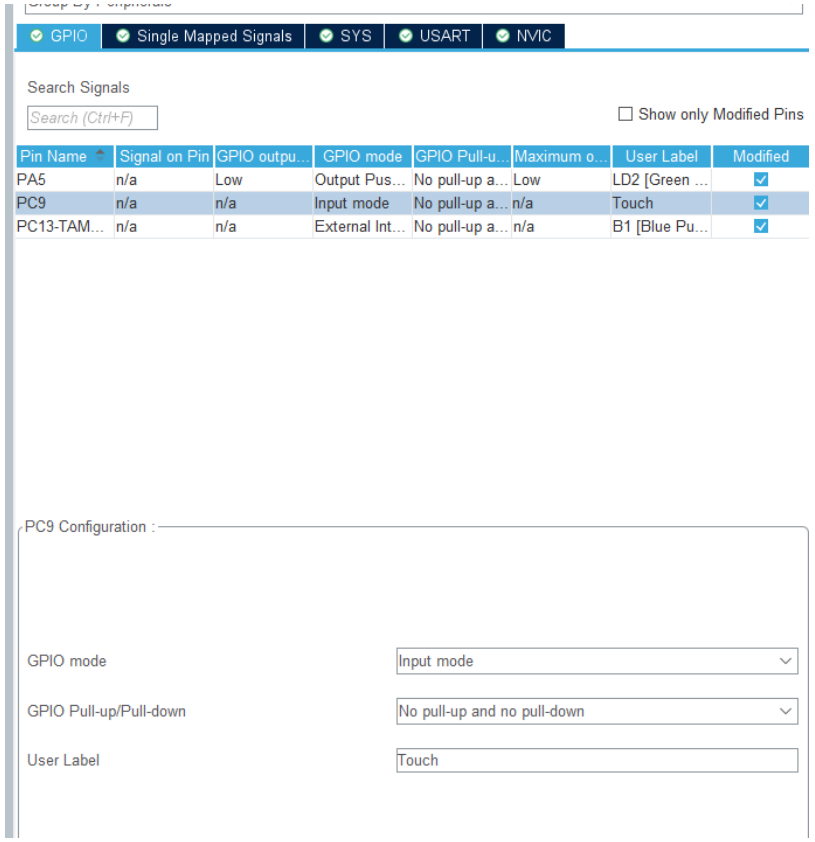




GPIO

>> GPIO 실습

- 단일 토글 모드 터치 센서, 터치로 인한 정전 용량 변화시 출력 핀 토글
- 터치에 따른 LED 동작 확인




```
98   while (1)
99   {
100    /* USER CODE END WHILE */
101
102    /* USER CODE BEGIN 3 */
103      if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_9))
104          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
105      else
106          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
107  }
108  /* USER CODE END 3 */
109 }
```

>> GPIO 과제

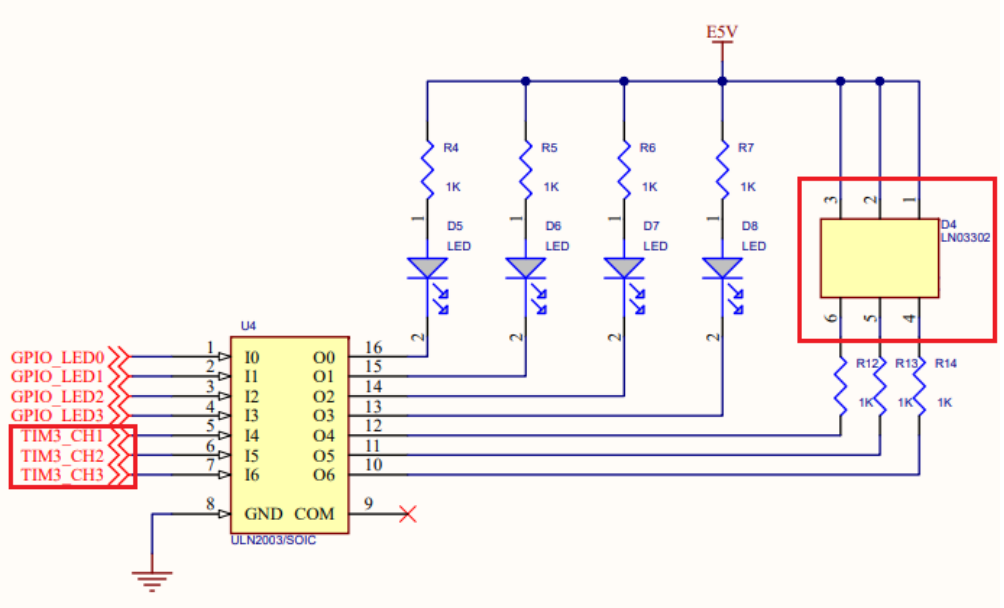
- GPIO 실습 과제 1
  - 보드 기본 LED와 버튼 사용
  - 처음 부팅 시 1초에 한번씩 LED가 점등
  - 버튼을 누른 시간에 한번씩 LED가 점등 (2초간 누르고 뗀 경우, 2초에 한번씩 점등)
  - HAL Delay는 Millisecond 단위
- (디버깅 과정에서 변수가 추적이 안될 경우 main함수 내에서 변수 선언 시도)
- 버튼 누르는 시간 측정에는 카운트 변수 혹은 HAL_GetTick 활용

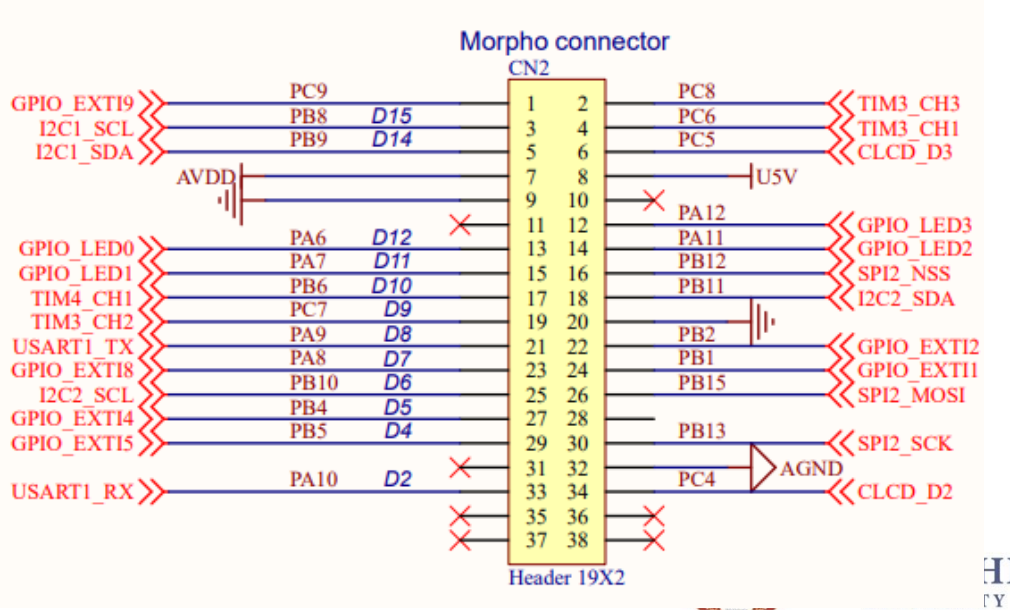
HAL_GetTick(); -> 현재 시스템 시간을 ms 단위로 리턴함

GPIO

>> GPIO 과제

- GPIO 실습 과제 2
  - 확장 보드 3색 LED 활용
  - 각 버튼에 따라 R, G, B 색 제어
  - 각 버튼을 한번 눌렀다 뗄때 마다 해당 색 LED 토글 제
  - 마지막 버튼을 누를경우 모든 LED 끄기





>> GPIO 과제

- GPIO 실습 과제 3
  - 확장 보드 LED, 버튼 4개 활용
  - 초기에 1번 LED가 1초에 한번씩 점등
  - 1번 버튼 클릭시 점등 LED가 좌측으로 이동(가장 왼쪽인 경우 가장 오른쪽으로 이동)
  - 2번 버튼 클릭시 점등 LED가 우측으로 이동 (가장 오른쪽인 경우 가장 왼쪽으로 이동)
  - 3번 버튼 클릭으로 점등 시간 간격 증가
  - 4번 버튼 클릭으로 점등 시간 간격 감소

EXTI

>> External Interrupt

- External Interrupt는 외부 핀(GPIO 핀 ...)에서 발생하는 이벤트를 감지하여 프로세서가 특정 동작을 수행하도록 하는 기능

- GPIO핀, 전원 감지, RTC(Real-Time Clock), USB 등 다양한 외부 입력 신호 감지 가능

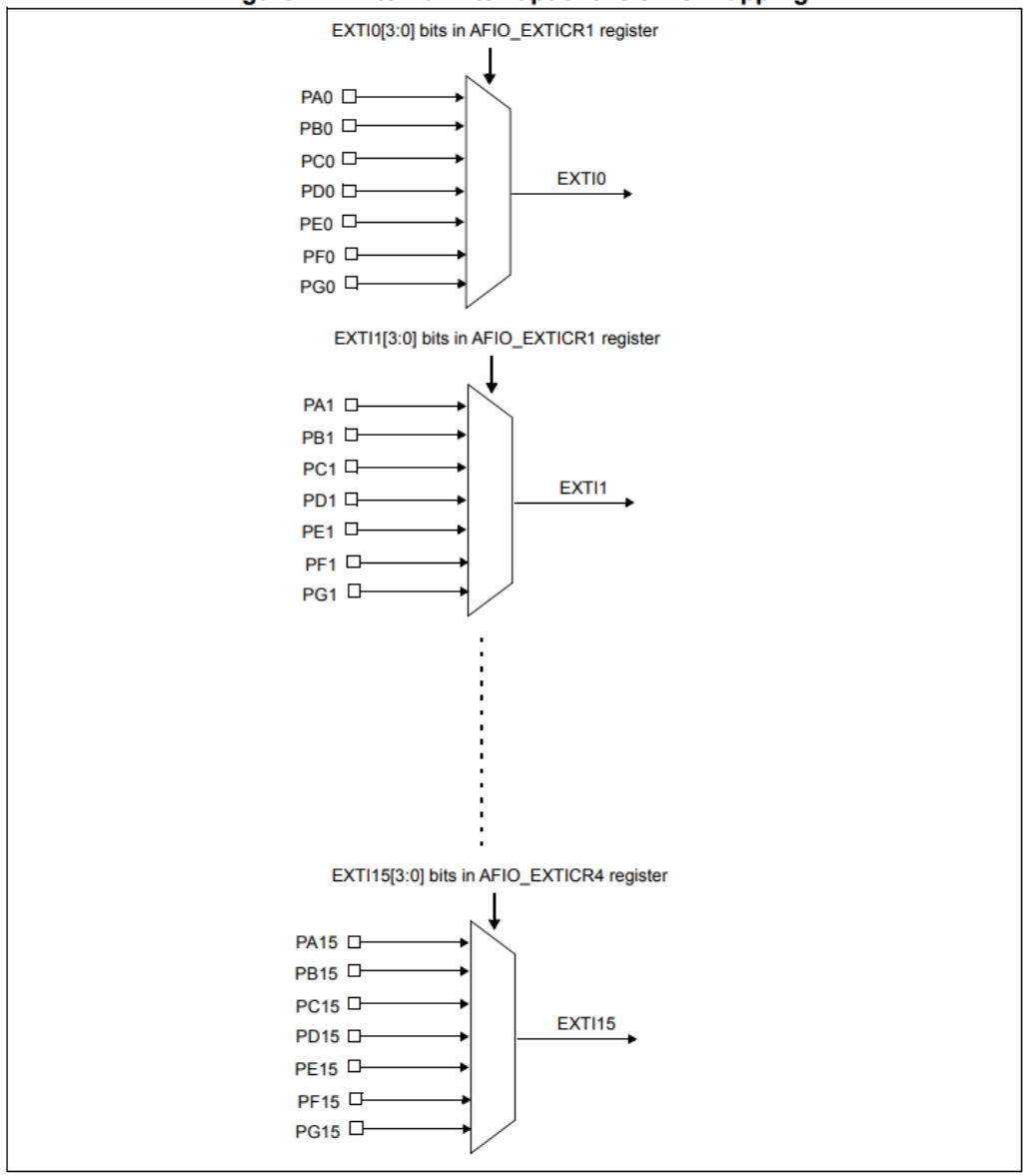
- EXTI 처리과정
  - 인터럽트 이벤트 감지, 인터럽트 요청 발생(Interrupt Request)
  - Nested Vectored Interrupt Controller(NVIC)가 Interrupt Vector Table을 기반으로 인터럽트의 우선순위 결정
  - IRQ Handler 실행을 통해 인터럽트 처리

- 일반적으로 사용하는 GPIO 인터럽트는 입력 신호의 변화를 감지함
  - Rising Edge: 상승 LOW -> HIGH 변화시 인터럽트 발생
  - Falling Edge: 하강 HIGH -> LOW 변화시 인터럽트 발생
  - Rising/Falling Edge: 양쪽 LOW <-> HIGH 변화시 인터럽트 발생

EXTI

>> External Interrupt

Figure 21. External interrupt/event GPIO mapping



The four other EXTI lines are connected as follows:

- EXTI line 16 is connected to the PVD output
- EXTI line 17 is connected to the RTC Alarm event
- EXTI line 18 is connected to the USB Wakeup event
- EXTI line 19 is connected to the Ethernet Wakeup event (available only in connectivity line devices)

- **EXTI0~15는 각 GPIO 포트의 0~15번 핀으로부터 외부 입력을 감지함**

- **EXTI16은 전원감지, EXTI17은 RTC Alarm, EXTI18은 USB Wakeup, EXTI19는 외부 통신 Wakeup event**

EXTI

>> External Interrupt

Table 63. Vector table for other STM32F10xxx devices

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000_0000
-	-3	fixed	Reset	Reset	0x0000_0004
-	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000_0008
-	-1	fixed	HardFault	All class of fault	0x0000_000C
-	0	settable	MemManage	Memory management	0x0000_0010
-	1	settable	BusFault	Prefetch fault, memory access fault	0x0000_0014
-	2	settable	UsageFault	Undefined instruction or illegal state	0x0000_0018
-	-	-	-	Reserved	0x0000_001C - 0x0000_002B
-	3	settable	SVCall	System service call via SWI instruction	0x0000_002C
-	4	settable	Debug Monitor	Debug Monitor	0x0000_0030
-	-	-	-	Reserved	0x0000_0034
-	5	settable	PendSV	Pendable request for system service	0x0000_0038
-	6	settable	SysTick	System tick timer	0x0000_003C
0	7	settable	WWDG	Window watchdog interrupt	0x0000_0040
1	8	settable	PVD	PVD through EXTI Line detection interrupt	0x0000_0044
2	9	settable	TAMPER	Tamper interrupt	0x0000_0048
3	10	settable	RTC	RTC global interrupt	0x0000_004C
4	11	settable	FLASH	Flash global interrupt	0x0000_0050
5	12	settable	RCC	RCC global interrupt	0x0000_0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000_0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000_005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000_0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000_0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000_0068
11	18	settable	DMA1_Channel1	DMA1 Channel1 global interrupt	0x0000_006C
12	19	settable	DMA1_Channel2	DMA1 Channel2 global interrupt	0x0000_0070
13	20	settable	DMA1_Channel3	DMA1 Channel3 global interrupt	0x0000_0074
14	21	settable	DMA1_Channel4	DMA1 Channel4 global interrupt	0x0000_0078
15	22	settable	DMA1_Channel5	DMA1 Channel5 global interrupt	0x0000_007C
16	23	settable	DMA1_Channel6	DMA1 Channel6 global interrupt	0x0000_0080
17	24	settable	DMA1_Channel7	DMA1 Channel7 global interrupt	0x0000_0084
18	25	settable	ADC1_2	ADC1 and ADC2 global interrupt	0x0000_0088
19	26	settable	USB_HP_CAN_TX	USB High Priority or CAN TX interrupts	0x0000_008C
20	27	settable	USB_LP_CAN_RX0	USB Low Priority or CAN RX0 interrupts	0x0000_0090
21	28	settable	CAN_RX1	CAN RX1 interrupt	0x0000_0094
22	29	settable	CAN_SCE	CAN SCE interrupt	0x0000_0098
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000_009C
24	31	settable	TIM1_BRK	TIM1 Break interrupt	0x0000_00A0
25	32	settable	TIM1_UP	TIM1 Update interrupt	0x0000_00A4
26	33	settable	TIM1_TRG_COM	TIM1 Trigger and Commutation interrupts	0x0000_00A8
27	34	settable	TIM1_CC	TIM1 Capture Compare interrupt	0x0000_00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000_00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000_00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000_00B8
31	38	settable	I2C1_EV	I^2C1 event interrupt	0x0000_00BC
32	39	settable	I2C1_ER	I^2C1 error interrupt	0x0000_00C0
33	40	settable	I2C2_EV	I^2C2 event interrupt	0x0000_00C4
34	41	settable	I2C2_ER	I^2C2 error interrupt	0x0000_00C8
35	42	settable	SPI1	SPI1 global interrupt	0x0000_00CC
36	43	settable	SPI2	SPI2 global interrupt	0x0000_00D0
37	44	settable	USART1	USART1 global interrupt	0x0000_00D4
38	45	settable	USART2	USART2 global interrupt	0x0000_00D8
39	46	settable	USART3	USART3 global interrupt	0x0000_00DC
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000_00E0
41	48	settable	RTCAlarm	RTC alarm through EXTI line interrupt	0x0000_00E4

- STM32F10xxx 시리즈의 Interrupt Vector Table
- EXTI0~4는 개별적인 EXTIx Handler를 가짐
- EXTI5~9는 EXTI9_5 Handler를 통해 하나로 묶임
- EXTI10~15도 동일하게 EXTI15_10 Handler로 다뤄짐

KYUNG HEE UNIVERSITY

EXTI

>> External Interrupt

RCC Mode and Configuration

Mode

High Speed Clock (HSE) Disable

Low Speed Clock (LSE) Disable

☐ Master Clock Output

GPIO | Single Mapped Signals | SYS | USART | NVIC

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin Na...	Signal on ...	GPIO out...	GPIO mode	GPIO Pull...	Maximum...	User Label	Modified
PA5	n/a	Low	Output Pu...	No pull-up...	Low	LD2 [Gree...	☑
PC13-TA...	n/a	n/a	External I...	No pull-up...	n/a	B1 [Blue ...	☑

PC13-TAMPER-RTC Configuration :

GPIO mode: External Interrupt Mode with Rising edge trigger detection

GPIO Pull-up/Pull-down: No pull-up and no pull-down

User Label: B1 [Blue PushButton]

GPIO | Single Mapped Signals | SYS | USART | NVIC

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
EXTI line[15:10] interrupts	☑	0	0

Categories | A->Z

System Core

- DMA
- GPIO
- IWDG
- NVIC
- RCC
- ✔ SYS
- WWDG

Analog

Timers

Connectivity

Computing

Middleware and Software Pac...

Configuration

NVIC | Code generation

Priority Group .. ☐ Sort by Premption Priority and Sub Priority ☐ Sort by interrupts names

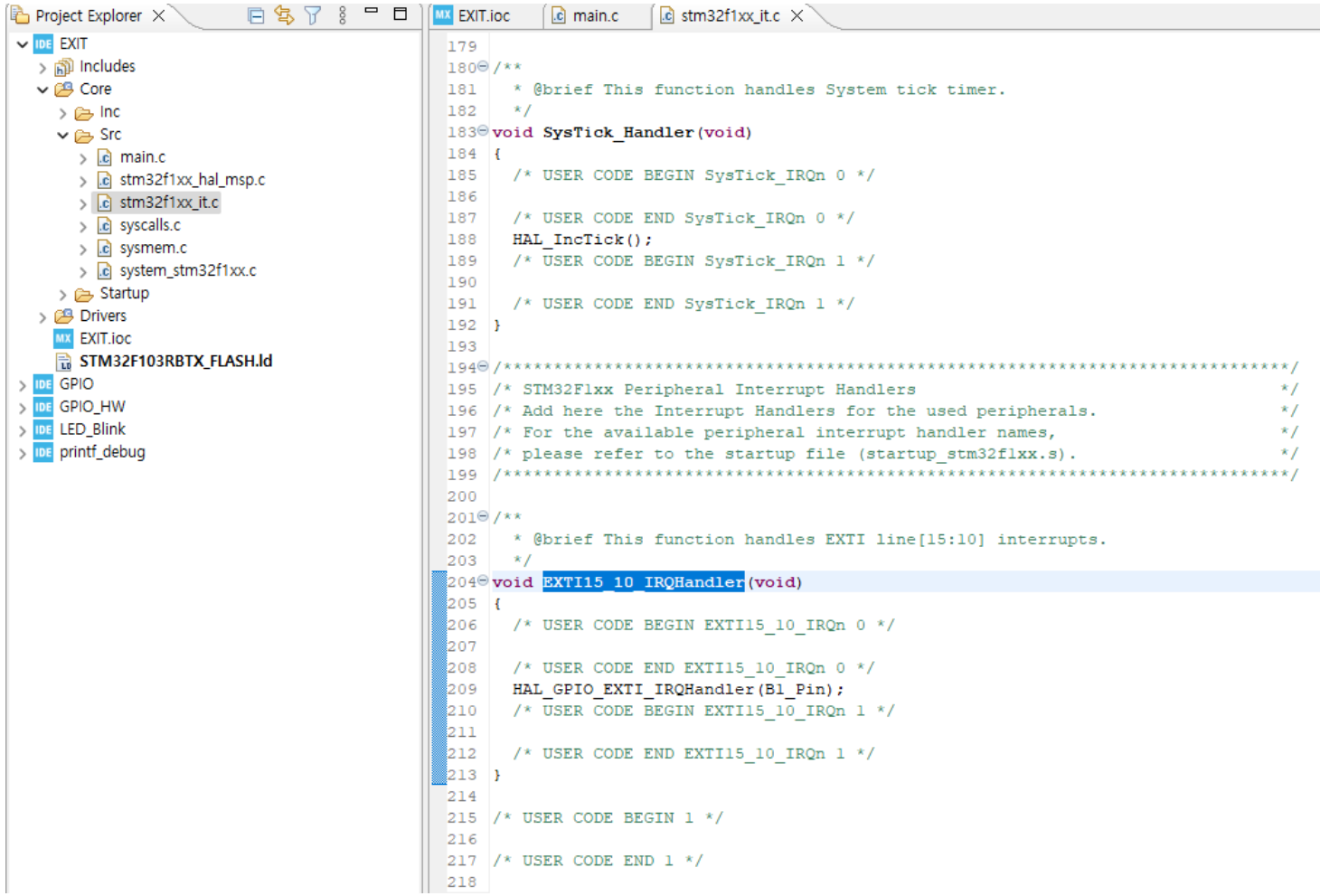
Search Show available interrupts ☑ Force DMA channels Inter

NVIC Interrupt Table	Enabled	Preemption Priority	Sub P
Non maskable interrupt	☑	0	0
Hard fault interrupt	☑	0	0
Memory management fault	☑	0	0
Prefetch fault, memory access fault	☑	0	0
Undefined instruction or illegal state	☑	0	0
System service call via SWI instruction	☑	0	0
Debug monitor	☑	0	0
Pendable request for system service	☑	0	0
Time base: System tick timer	☑	0	0
PVD interrupt through EXTI line 16	☐	0	0
Flash global interrupt	☐	0	0
RCC global interrupt	☐	0	0
USART2 global interrupt	☐	0	0
EXTI line[15:10] interrupts	☑	0	0

EXTI

>> External Interrupt

- EXTI 설정을 통해 NVIC IVT에 해당 핀에 대한 인터럽트가 추가됨
- 코드 생성 후 stm32f1xx_it.c에 해당 EXTI IRQ Handler가 생성
- 사용하는 GPIO 핀은 PC13, 따라서 EXTI15_10 Handler를 사용함
- `HAL_GPIO_EXTI_IRQHandler`정의 코드로 이동



함수를 Ctrl+좌클릭/함수 선택 후 F3/함수 우클릭 후 Open Declaration으로 해당 함수 정의부분 으로 이동 가능

EXTI

>> External Interrupt

- EXTI Pending Register를 통해 현재 인터럽트가 해당 인터럽트인지 확인
- 이후 Callback함수를 호출해 인터럽트 처리
- Callback 함수는 weak로 선언되어 사용자가 인터럽트 처리를 직접 설정 가능

10.3.6 Pending register (EXTI_PR)

Address offset: 0x14
Reset value: undefined

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												PR19	PR18	PR17	PR16
												rc_w1	rc_w1	rc_w1	rc_w1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:20 Reserved, must be kept at reset value (0).

Bits 19:0 **PRx:** Pending bit
0: No trigger request occurred
1: selected trigger request occurred
This bit is set when the selected edge event arrives on the external interrupt line. This bit is cleared by writing a '1' into the bit.

Note: Bit 19 is used in connectivity line devices only and is reserved otherwise.

stm32f1xx_hal_gpio.c

```c
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  /* EXTI line interrupt detected */
  if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin);
  }
}

/**
  * @brief  EXTI line detection callbacks.
  * @param  GPIO_Pin: Specifies the pins connected EXTI line
  * @retval None
  */
__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(GPIO_Pin);
  /* NOTE: This function Should not be modified, when the callback is needed,
           the HAL_GPIO_EXTI_Callback could be implemented in the user file
   */
}
```

```c
#define __HAL_GPIO_EXTI_GET_IT(__EXTI_LINE__) (EXTI->PR & (__EXTI_LINE__))

/**
  * @brief  Clears the EXTI's line pending bits.
  * @param  __EXTI_LINE__: specifies the EXTI lines to clear.
  *         This parameter can be any combination of GPIO_PIN_x where x can b
  * @retval None
  */
#define __HAL_GPIO_EXTI_CLEAR_IT(__EXTI_LINE__) (EXTI->PR = (__EXTI_LINE__))
```

KYUNG HEE UNIVERSITY

EXTI

>> External Interrupt

- main.c에 `HAL_GPIO_EXTI_Callback`함수 재정의
- while 부분에는 별도의 코드 작성이 없음
- 버튼에 따른 LED 동작 확인
- 인터럽트의 Callback을 통해 LED 제어

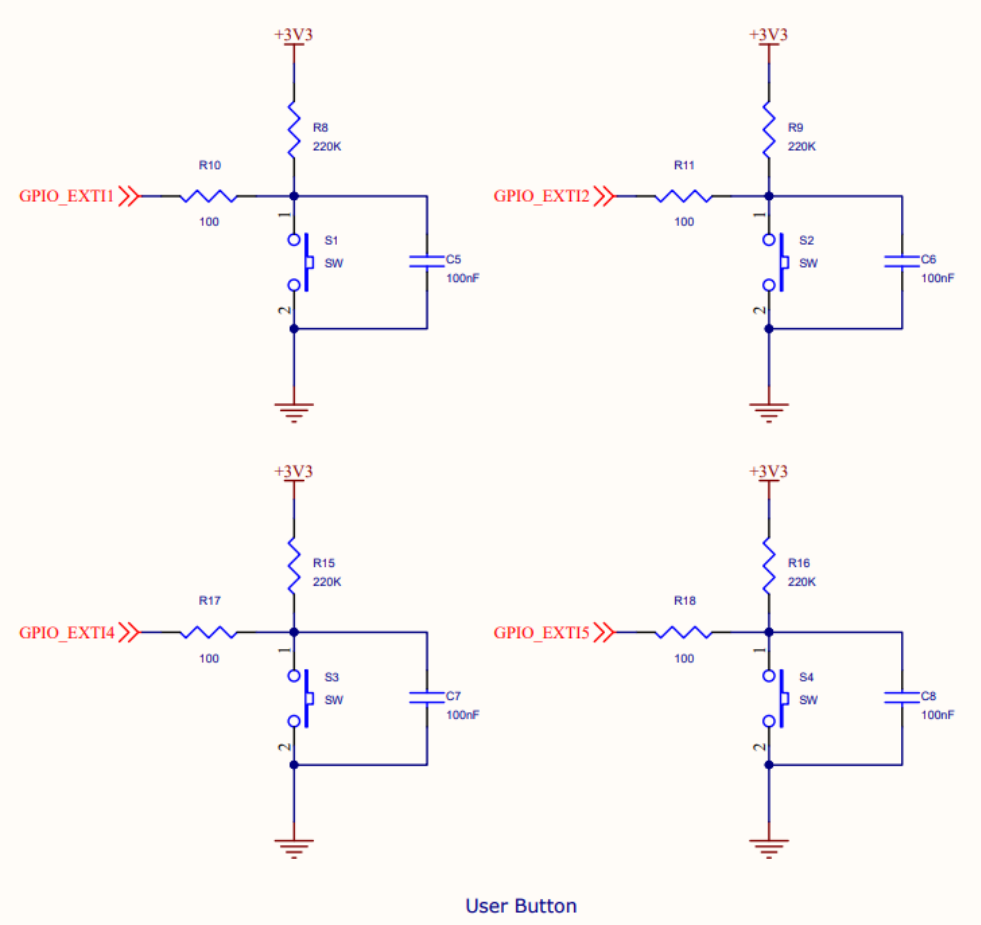
main.c

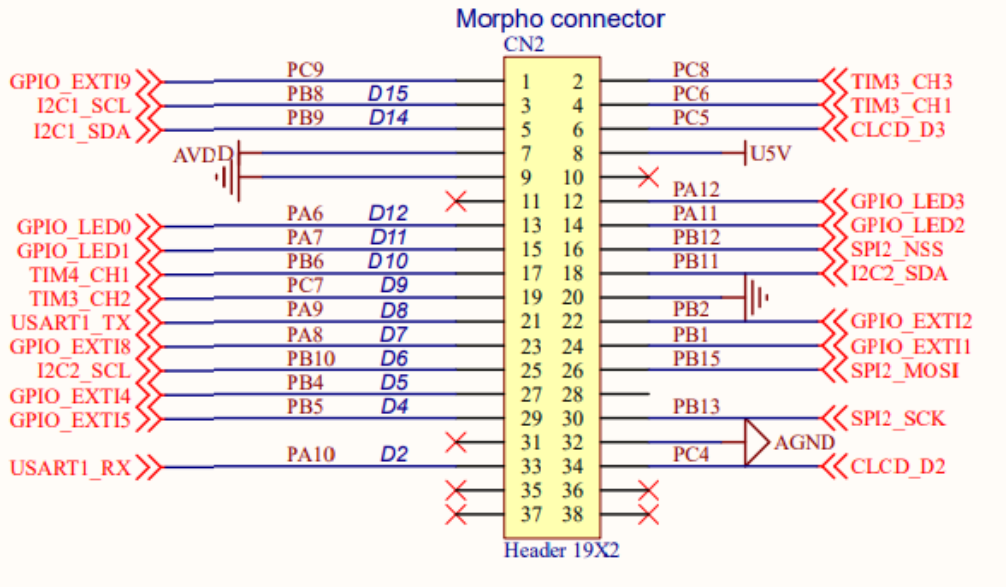
```c
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    switch (GPIO_Pin) {
    case B1_Pin:
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
        break;

    default:
        ;
    }
}
/* USER CODE END 4 */
```

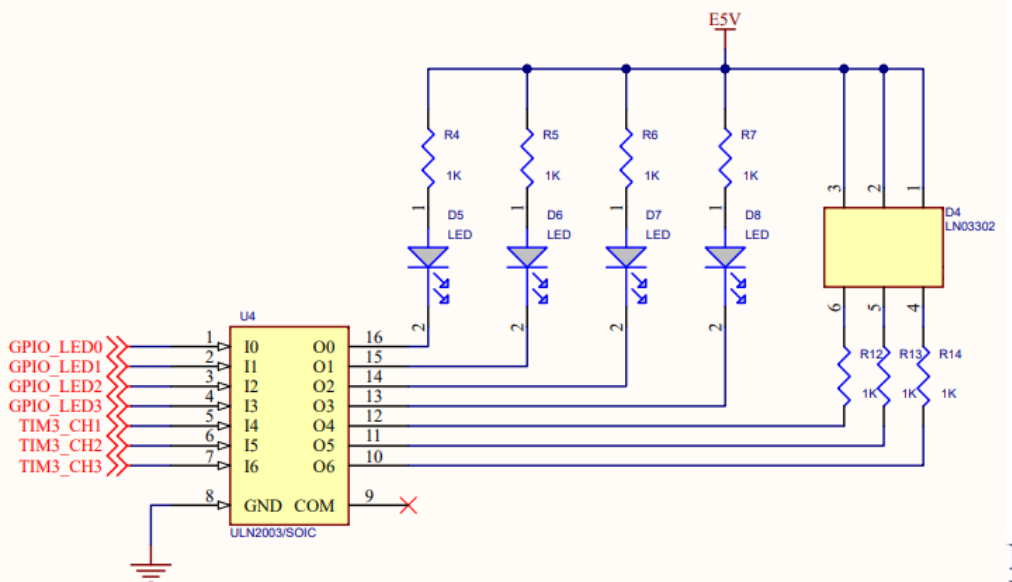
EXTI

>> EXTI 실습

- **확장 보드 실습 - EXTI**







EXTI

>> EXTI 실습

- PA6, 7, 11,12 -> GPIO Output
- PB1, 2, 4, 5 -> GPIO EXTI
- EXTI1, 2, 4, 9_5 -> Rising/Falling edge trigger

GPIO | Single Mapped Signals | SYS | USART | NVIC

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin Name	Signal on Pin	GPIO out...	GPIO mode	GPIO Pull-...	Maximum ou...	User Label	Modified
PA6	n/a	Low	Output Push Pull	No pull-up ...	Low	LED1	☑
PA7	n/a	Low	Output Push Pull	No pull-up ...	Low	LED2	☑
PA11	n/a	Low	Output Push Pull	No pull-up ...	Low	LED3	☑
PA12	n/a	Low	Output Push Pull	No pull-up ...	Low	LED4	☑
PB1	n/a	n/a	External Interrupt ...	No pull-up ...	n/a	B1	☑
PB2	n/a	n/a	External Interrupt ...	No pull-up ...	n/a	B2	☑
PB4	n/a	n/a	External Interrupt ...	No pull-up ...	n/a	B3	☑
PB5	n/a	n/a	External Interrupt ...	No pull-up ...	n/a	B4	☑

PB1 Configuration :

GPIO mode: External Interrupt Mode with Rising/Falling edge trigger detection

GPIO Pull-up/Pull-down: No pull-up and no pull-down

User Label: B1

GPIO | Single Mapped Signals | SYS | USART | NVIC

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
EXTI line1 interrupt	☑	0	0
EXTI line2 interrupt	☑	0	0
EXTI line4 interrupt	☑	0	0
EXTI line[9:5] interrupts	☑	0	0

EXTI

>> EXTI 실습

- Preemption Priority(선점 우선순위)
  - 낮은 숫자일수록 높은 우선순위
  - 낮은 우선순위 인터럽트를 중단하고 높은 우선순위 인터럽트를 먼저 실행

- Sub Priority(서브 우선순위)
  - 낮은 숫자일수록 높은 우선순위
  - 같은 Preemption Priority를 가진 인터럽트 간의 우선순위 결정

GPIO | Single Mapped Signals | SYS | USART | NVIC

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
EXTI line1 interrupt	☑	0	0
EXTI line2 interrupt	☑	0	0
EXTI line4 interrupt	☑	0	0
EXTI line[9:5] interrupts	☑	0	0

EXTI

>> EXTI 실습

- main.c의 `MX_GPIO_Init`에서 NVIC 설정 확인

- PB1, 2, 4, 5 GPIO_EXTI 트리거가 `GPIO_MODE_IT_RISING_FALLING`로 설정됨

- EXTI1, EXTI2, EXTI3, EXTI9_5의 우선순위 설정

main.c

```c
183 static void MX_GPIO_Init(void)
184 {
185   GPIO_InitTypeDef GPIO_InitStruct = {0};
186 /* USER CODE BEGIN MX_GPIO_Init_1 */
187 /* USER CODE END MX_GPIO_Init_1 */
188
189   /* GPIO Ports Clock Enable */
190   __HAL_RCC_GPIOC_CLK_ENABLE();
191   __HAL_RCC_GPIOD_CLK_ENABLE();
192   __HAL_RCC_GPIOA_CLK_ENABLE();
193   __HAL_RCC_GPIOB_CLK_ENABLE();
194
195   /*Configure GPIO pin Output Level */
196   HAL_GPIO_WritePin(GPIOA, LED1_Pin|LED2_Pin|LED3_Pin|LED4_Pin, GPIO_PIN_RESET);
197
198   /*Configure GPIO pins : LED1_Pin LED2_Pin LED3_Pin LED4_Pin */
199   GPIO_InitStruct.Pin = LED1_Pin|LED2_Pin|LED3_Pin|LED4_Pin;
200   GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
201   GPIO_InitStruct.Pull = GPIO_NOPULL;
202   GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
203   HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
204
205   /*Configure GPIO pins : B1_Pin B2_Pin B3_Pin B4_Pin */
206   GPIO_InitStruct.Pin = B1_Pin|B2_Pin|B3_Pin|B4_Pin;
207   GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING_FALLING;
208   GPIO_InitStruct.Pull = GPIO_NOPULL;
209   HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
210
211   /* EXTI interrupt init*/
212   HAL_NVIC_SetPriority(EXTI1_IRQn, 0, 0);
213   HAL_NVIC_EnableIRQ(EXTI1_IRQn);
214
215   HAL_NVIC_SetPriority(EXTI2_IRQn, 0, 0);
216   HAL_NVIC_EnableIRQ(EXTI2_IRQn);
217
218   HAL_NVIC_SetPriority(EXTI4_IRQn, 0, 0);
219   HAL_NVIC_EnableIRQ(EXTI4_IRQn);
220
221   HAL_NVIC_SetPriority(EXTI9_5_IRQn, 0, 0);
222   HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
223
224 /* USER CODE BEGIN MX_GPIO_Init_2 */
225 /* USER CODE END MX_GPIO_Init_2 */
226 }
```

EXTI

>> EXTI 실습

- 스위치를 누를 때(Falling)와 뗄 때(Rising) 인터럽트가 트리거되어 Callback 함수 호출

- Rising Edge: 인터럽트 발생시 핀 상태가 1, LED 켜기

- Falling Edge : 인터럽트 발생시 핀 상태가 0, LED 끄기

main.c

```c
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin))
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
    else
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);

    if (HAL_GPIO_ReadPin(B2_GPIO_Port, B2_Pin))
        HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_RESET);
    else
        HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_SET);

    if (HAL_GPIO_ReadPin(B3_GPIO_Port, B3_Pin))
        HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_RESET);
    else
        HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_SET);

    if (HAL_GPIO_ReadPin(B4_GPIO_Port, B4_Pin))
        HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_RESET);
    else
        HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_SET);
}
/* USER CODE END 4 */
```

EXTI

>> EXTI 실습

Categories | A->Z

System Core
- DMA
- GPIO
- IWDG
- NVIC
- RCC
- SYS
- WWDG

Analog

Timers

Connectivity

Computing

Middleware and Software Pac...

Configuration

NVIC | Code generation

Priority Group: 4 bits fo... | Sort by Premption Priority and Sub Priority | Sort by interrupts names

Search | Show: available interrupts | Force DMA channels Interrupts

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
Non maskable interrupt	☑	0	0
Hard fault interrupt	☑	0	0
Memory management fault	☑	0	0
Prefetch fault, memory access fault	☑	0	0
Undefined instruction or illegal state	☑	0	0
System service call via SWI instruction	☑	0	0
Debug monitor	☑	0	0
Pendable request for system service	☑	0	0
Time base: System tick timer	☑	0	0
PVD interrupt through EXTI line 16	☐	0	0
Flash global interrupt	☐	0	0
RCC global interrupt	☐	0	0
EXTI line1 interrupt	☑	4	0
EXTI line2 interrupt	☑	3	0
EXTI line4 interrupt	☑	2	0
EXTI line[9:5] interrupts	☑	1	0
USART2 global interrupt	☐	0	0

EXTI

>> EXTI 실습

- 인터럽트 발생시 해당 핀의 LED를 5번 점등
- 버튼 1번이 가장 낮은 우선순위, 버튼 4번이 가장 높은 우선순위를 가짐
- 동일한 Callback함수 내에서 switch 문을 통해 구별
- 따라서 버튼 각각의 인터럽트에 대한 작업(해당 되는 버튼의 LED 점등)이 진행됨

```c
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    switch (GPIO_Pin) {
    case B1_Pin:
        for (int i = 0; i < 10; i++) {
            HAL_GPIO_TogglePin(GPIOA, LED1_Pin);
            HAL_Delay(250);
        }
        break;

    case B2_Pin:
        for (int i = 0; i < 10; i++) {
            HAL_GPIO_TogglePin(GPIOA, LED2_Pin);
            HAL_Delay(250);
        }
        break;

    case B3_Pin:
        for (int i = 0; i < 10; i++) {
            HAL_GPIO_TogglePin(GPIOA, LED3_Pin);
            HAL_Delay(250);
        }
        break;

    case B4_Pin:
        for (int i = 0; i < 10; i++) {
            HAL_GPIO_TogglePin(GPIOA, LED4_Pin);
            HAL_Delay(250);
        }
        break;
    default:
        ;
    }
}
/* USER CODE END 4 */
```

>> EXTI 실습

- 버튼 1번부터 4번까지 순서대로 눌러 LED 상태 확인
- 버튼 4번부터 1번까지 순서대로 눌러 LED 상태 확인
- 여러가지 순서 조합으로 버튼 눌러 LED 상태 확인
- 버튼 여러번 눌러 LED 점등 횟수 확인

EXTI

>> EXTI 실습

- NVIC를 통해 EXIT와 다른 인터럽트의 우선순위 조절을 사용자가 원하는대로 설정이 가능함

- 우선순위에 따라서 작업의 실행 순서가 변경됨.
  - 낮은 우선순위 인터럽트 작업은 중단되고 높은 우선순위 인터럽트 작업이 끼어들어 실행됨
  - 이후 높은 우선순위 인터럽트 처리가 완료된 후 낮은 우선순위 인터럽트가 처리됨

- Cortex M3의 NVIC의 Tail-Chaining과 Late Arrival 기술

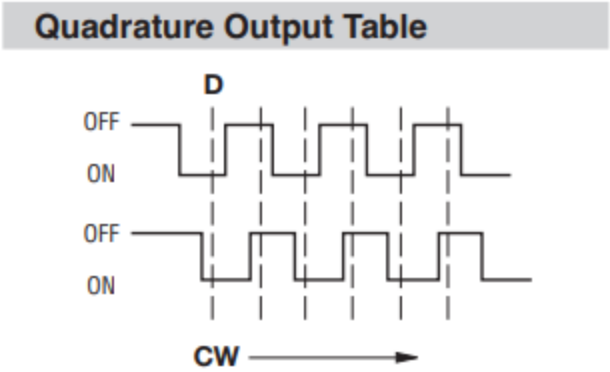
EXTI line1 interrupt	☑	4	0
EXTI line2 interrupt	☑	3	0
EXTI line4 interrupt	☑	2	0
EXTI line[9:5] interrupts	☑	1	0

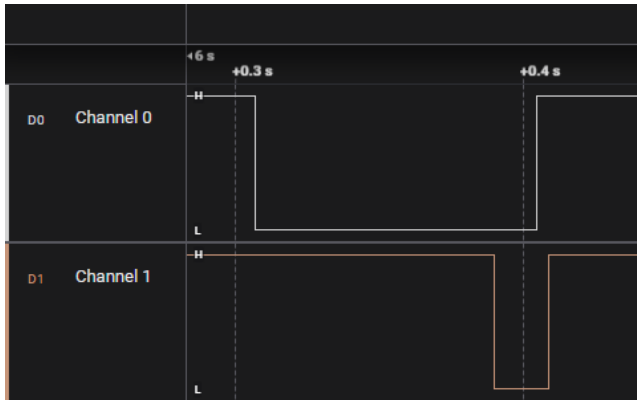
```
/* EXTI interrupt init*/
HAL_NVIC_SetPriority(EXTI1_IRQn, 4, 0);
HAL_NVIC_EnableIRQ(EXTI1_IRQn);

HAL_NVIC_SetPriority(EXTI2_IRQn, 3, 0);
HAL_NVIC_EnableIRQ(EXTI2_IRQn);

HAL_NVIC_SetPriority(EXTI4_IRQn, 2, 0);
HAL_NVIC_EnableIRQ(EXTI4_IRQn);

HAL_NVIC_SetPriority(EXTI9_5_IRQn, 1, 0);
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
```

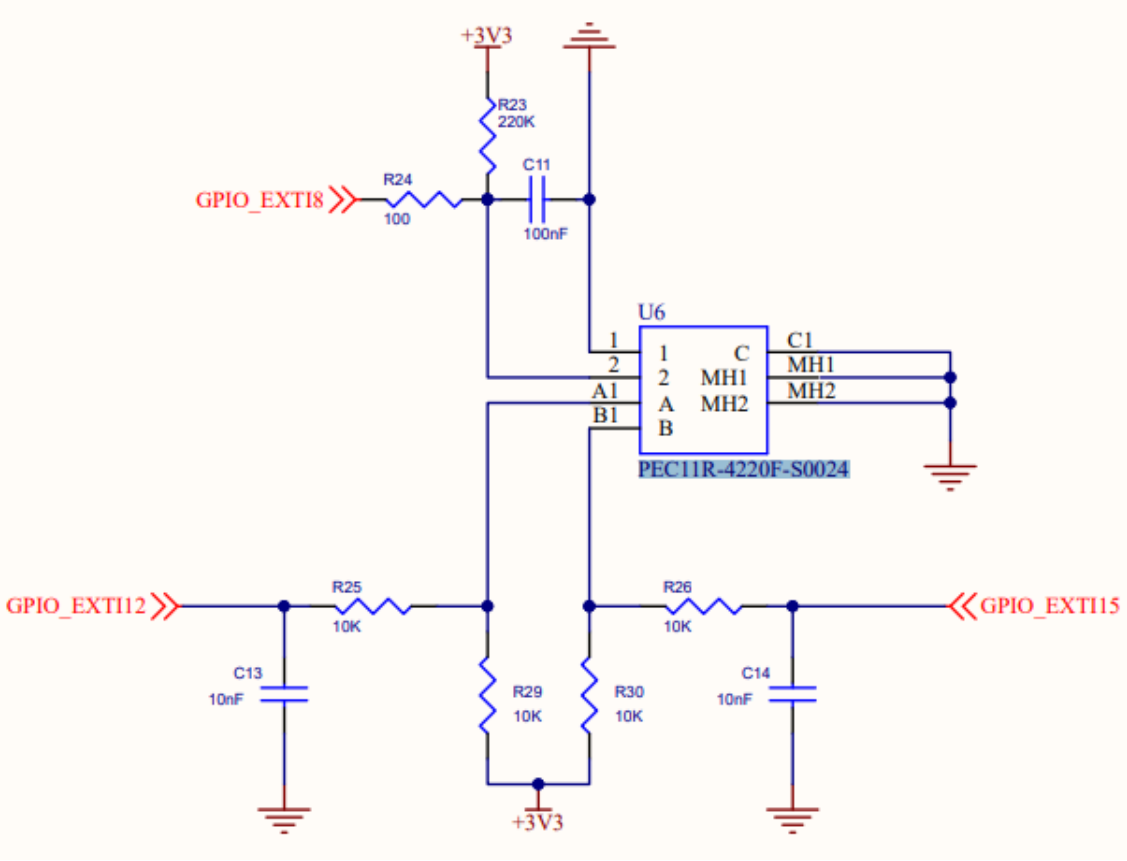
EXTI

>> EXTI 실습

- 확장 보드 실습 – 로터리 엔코더
- PA8 – Falling / PA15 – Rising / PC12 – Rising
- 시계 방향인 경우 A포트가 B 포트보다 먼저 Rising
- 반시계 방향인 경우 B포트가 A포트보다 먼저 Rising





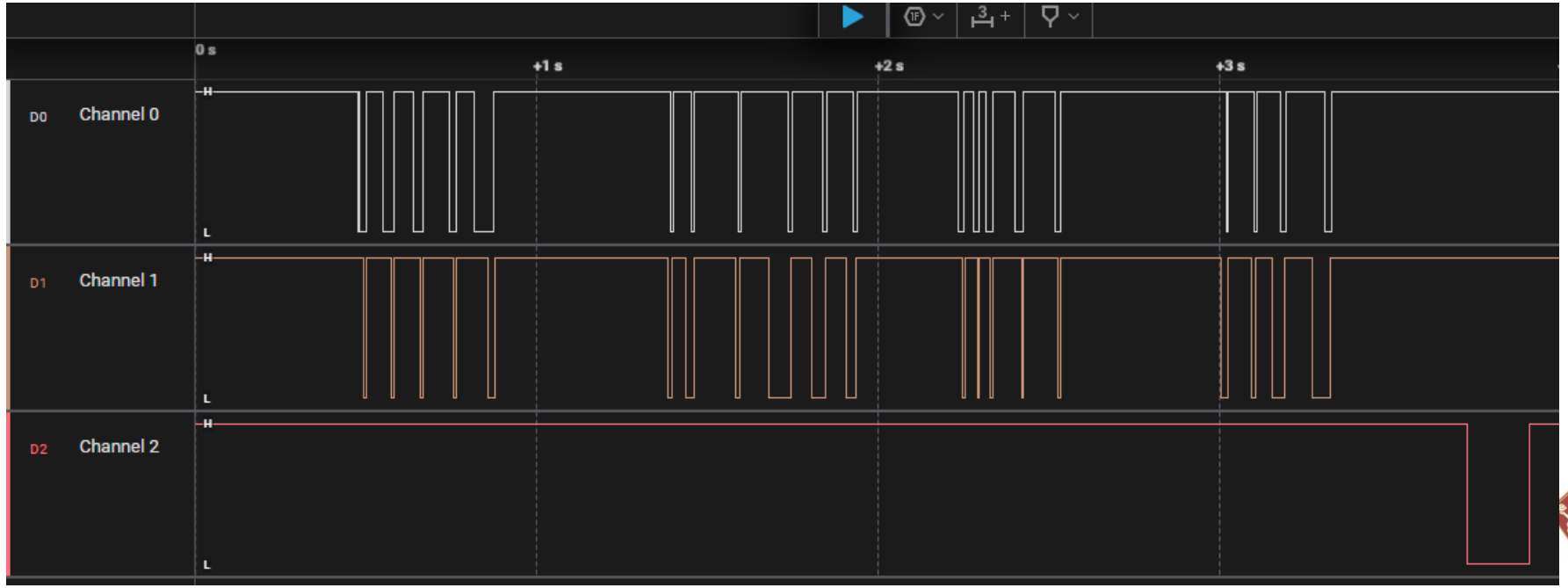


KYUNG HEE UNIVERSITY

EXTI

>> EXTI 실습

- A 포트 – PC12: Rising 트리거시 B포트가 LOW 인 경우 시계방향(CW) 카운트
- B 포트 – PA15 : Rising 트리거시 A포트가 LOW 인 경우 반시계방향(CCW) 카운트
- 푸시 버튼 – PA8: 카운트 리셋
- 로직 분석기로 로터리 엔코더 신호 분석 해볼것



KYUNG HEE UNIVERSITY

EXTI

>> EXTI 실습

- **UART printf 코드 추가하여 활용**

```c
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
#ifdef __GNUC__
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
#endif

PUTCHAR_PROTOTYPE
{
    if (ch == '\n')
        HAL_UART_Transmit(&huart2, (uint8_t*) "\r", 1, 0xFFFF);
    HAL_UART_Transmit(&huart2, (uint8_t*) &ch, 1, 0xFFFF);

    return ch;
}
/* USER CODE END 0 */
```

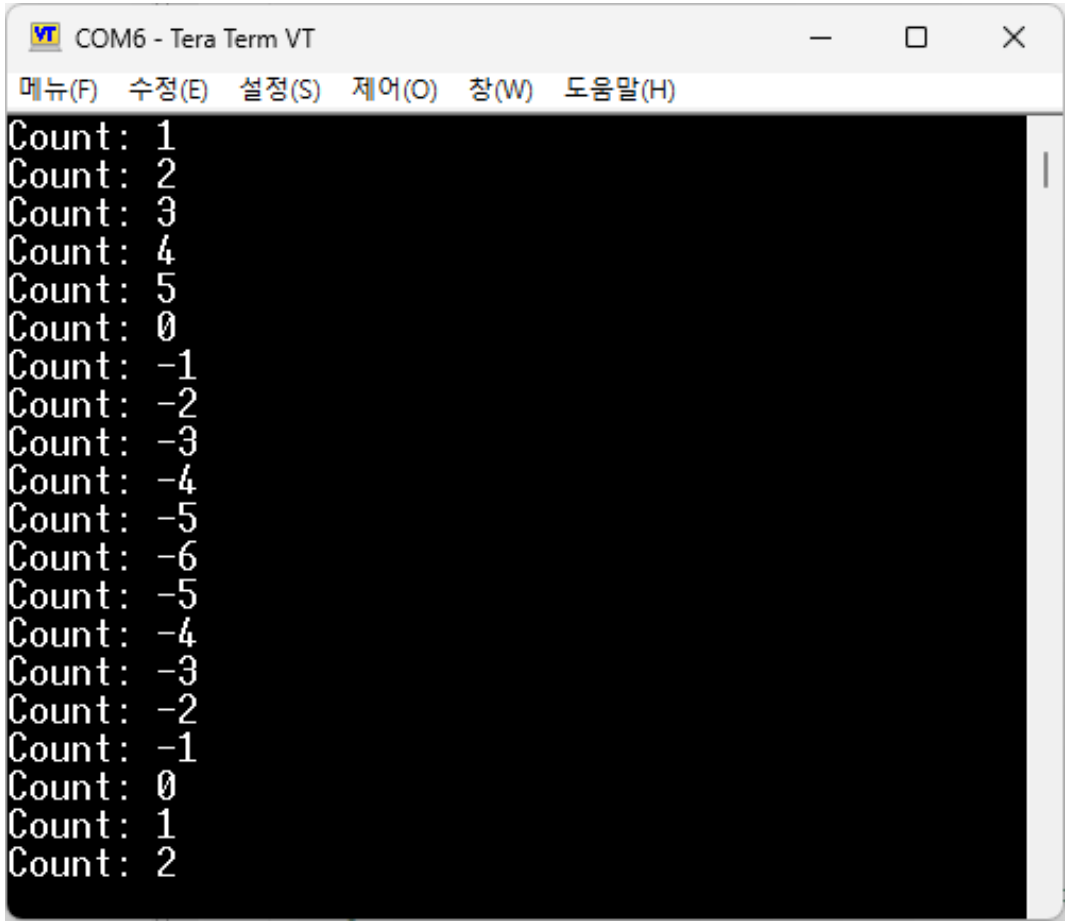
```c
  int prevCnt = 0;

  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
      if(prevCnt != EncCnt) {
          prevCnt = EncCnt;
          printf("Count: %d\n", EncCnt);
      }
  }
  /* USER CODE END 3 */
}
```

```c
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    switch(GPIO_Pin) {
    case(GPIO_PIN_8):
        EncCnt = 0;
        break;

    case(port_A_Pin):
        if(!HAL_GPIO_ReadPin(port_B_GPIO_Port, port_B_Pin))
            EncCnt++;
        break;

    case(port_B_Pin):
        if(!HAL_GPIO_ReadPin(port_A_GPIO_Port, port_A_Pin))
            EncCnt--;
        break;

    default:
        ;
    }
}
/* USER CODE END 4 */
```

EXTI

>> EXTI 실습

- Tera Term 실행하여 로터리 엔코더 카운터 출력 확인
- 시계 방향 회전시 카운트 증가/반시계 방향 회전시 카운트 감소
- 노브 푸시 버튼 클릭시 카운터 리셋
- CubeIDE의 디버깅 모드에서도 확인해보기
  - printf부분에 breakpoint 설정후 디버깅 시도



EXTI

>> EXTI 실습

EXTI_Encoder.ioc | main.c | stm32f1xx_it.c | main.h | system_stm32f1x | startup_stm32f1

```c
  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  int prevCnt = 0;

  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
      if(prevCnt != EncCnt) {
          prevCnt = EncCnt;
          printf("Count: %d\n", EncCnt);
      }
  }
  /* USER CODE END 3 */
}
```

Variabl | Breakp | Express | Registe | Live Ex | SFRs

Name	Type	Value
prevCnt	int	36

>> EXTI 과제

- EXTI 실습 과제 1
  - 확장보드 LED 4개 사용
  - R -> G -> B -> Y -> R -> … 주기적으로 LED 켜기 (하나의 LED만 켜질것)
  - 1번 버튼: 속도 증가
  - 2번 버튼: 속도 감소
  - 3번 버튼: 켜지는 순서 반대 변경
  - 4번 버튼: 현재 상태 정지 및 동작 재개

>> EXTI 과제

- EXTI 실습 과제 2
  - 모든 LED OFF 상태로 시작
  - 버튼 1번을 1초 이상 누른 경우 설정 모드 진입 (모든 LED가 1초 주기로 점등)
  - 버튼 2번을 통해 좌측 LED로 이동(가장 좌측인 경우 이동 x)
  - 버튼 3번을 통해 우측 LED로 이동(가장 우측인 경우 이동 x)
  - 버튼 4번을 통해 LED 선택/선택 해제 (선택한 LED는 0.5초 주기로 점등, 해제시 기존대로 1초 주기 점등)
  - 버튼 1번을 다시 길게 누를 경우 설정 모드 이탈 및 선택된 LED를 일정 주기로 점등

HAL_GetTick(); -> 현재 시스템 시간을 ms 단위로 리턴함

Thank you

Department of Electronic Engineering,
Kyung Hee University, 1732, Deogyeong-Daero,
Giheung-gu, Yongin-si, Gyeonggi-Do, 17104,
Republic of Korea
Web: https://khu-hri.weebly.com/

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY